e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

# Enterprise-grade AI Powered Placement Readiness and Multimodal Interview Platform

**Chandana M N[1], Chandana M P[2], Harshini R B[3] , Harshitha B M[4], Shilpa S[5]**

[1,2,3,4] Students , Department of Computer Science & Engineering, Kalpataru Institute of Technology Tiptur-572201,Karnataka ,India

[5] Assistant Professor, Department of Computer Science & Engineering, Kalpataru Institute of Technology Tiptur-572201, Karnataka, India

Affiliated with Visvesvaraya Technological University (VTU), Belagavi

## ABSTRACT

Undergraduate Engineering graduates face high attrition in campus placements due to the use of automated systems like Applicant Tracking Systems (ATS), nervousness during technical interviews and failure to explain algorithmic complexities. We propose Career Navigator an enterprise grade platform for candidate placement readiness. We make the following contributions: (1) Deterministic ATS resume parser based on ligature normalisation, DJB2 hashing and category-based scoring to provide objective and bias-free scoring ($R2 = 0.892, p < 0.001$). (2) Low-latency conversation based mock interview using Google Gemini 2.5 Flash and Server-Sent Events (SSE) to provide low token-latency of 342ms (78.4% faster). (3) Client-side voice telemetry using HTML5 Web Audio API for Words Per Minute (WPM) calculation with 98.9% accuracy. (4) In-browser algorithm lab using Monaco editor and static Abstract Syntax Tree (AST) analysis to determine algorithmic complexities with 95.0% accuracy.

Extensive load-testing is performed to show that server can handle over 720 req/s with 500 virtual users. Automated testing shows 100% pass percentage for 78 tests.

**Keywords:** Applicant Tracking Systems, Generative Artificial Intelligence, Server-Sent Events, Voice Telemetry, Abstract Syntax Trees, Algorithm Complexities, Web Apps.

## I. INTRODUCTION

The way companies recruit Software Engineers have evolved due to high numbers of applications. Every role in an enterprise company will have thousands of resumes coming in. To deal with this large volume of applicants companies utilise automated systems to shortlist candidates. These include Applicant Tracking Systems (ATS), automated coding testing and asynchronous video interviews [1], [2].

This work was funded by the Institutional Innovation and Research Council, Department of Computer Science & Engineering, Kalpataru Institute of Technology under Grant KIT/RIC/2026-27/GR-042.

There is a clear divide between the knowledge students gain from studying engineering and what happens during the placement process [27]. Although students have knowledge of algorithms, system programming and other concepts learned within the computer science degree they often fail to pass initial screening. This is often down to:

1) Typos and Formatting Issues:

Many resumes will have complex layouts with various fonts and images. Many Commercial ATS parsers will not be able to read these as they use simple text readers. This means many candidates will be rejected without being reviewed by a human [20].

2) Nervousness in Conversation: Candidates will often speak too fast (> 170 words per minute), have long pauses and high levels of filler in their answers. This leads to a poor impression from recruiters [2].

3) Lack of Explanation of Algorithms: Candidates will often write correct functions but fail to explain the time and space complexity of their algorithm (O-notation) [5].

The current options available on the market are limited. Commercial ATS parsers are expensive and provide no explanation of how they score candidates [8]. Interview coaching platforms are expensive with tutors costing more than $150/hour. Algorithm practice platforms also fail to show the complexity of algorithms [27].

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

In this work we develop Career Navigator an enterprise grade placement preparation platform. This includes:

- Deterministic ATS parser using ligature normalisation, 32-bit DJB2 hashing and keyword scoring based on categories.
- Low-latency conversation interview using Google Gemini 2.5 Flash and Server-Sent Events (SSE). This provides very low token latency (< 400ms).
- Voice analytics using the W3C Web Audio API and Speech Recognition interfaces in the browser. This provides voice metrics without sending voice data.
- Static Abstract Syntax Tree (AST) analysis to determine algorithmic complexities. This does this by looking at the syntax tree rather than run-time.
- Testing with 50 real resumes shows a high correlation (R2 = 0.892) with senior technical recruiters.

The rest of this paper is structured as follows. Section II looks at related work. Section III describes the systems architecture and security features. Section IV provides the mathematical formulations of the algorithms. Section V provides the results. Section VI looks at pedagogical impacts and limitations. Section VII concludes.

## II. RELATED WORK

Automated recruiting uses Natural Language Processing, Speech Processing, Generative Artificial Intelligence and Static Analysis.

### A. Parsing Irregular Documents

Many early resume parsers used regular expressions and Context-Free Grammars. Anderson et al. [1] looked at using Conditional Random Fields and Bidirectional LSTMs for Named Entity Recognition in technical resumes. This provided high levels of accuracy when parsing single-column resumes but failed with multi-column resumes (error rate > 25%). Xu et al. [20] developed LayoutLM which showed that to understand documents you need to look at both the text and position of text in 2D space. However, LayoutLM has a large inference cost meaning it is not feasible to implement in the browser. Career Navigator provides a deterministic resume parser using multiple passes of regular expressions. This uses ligature normalisation and aliasing to avoid large costs of using Neural networks to parse documents.

### B. Generative Large Language Models

The introduction of the Transformer [3] and foundation models [4] has changed the way conversation based tutoring works. Higher education provides much higher learning acceleration (up to 3.2x) with formative feedback than summative [22]. However, Generative Large Language Models can cause issues with scoring. This can include hallucinations and variance in scoring [21]. Career Navigator provides strict JSON formatting rules for Gemini 2.5 Flash [13], [14].

### C. Streaming with Server-Sent Events

Generating generative feedback requires streaming. Radboud et al. [9] looked at different ways to generate generative tokens and found that using Server-Sent Events had 38% less overhead than using WebSockets via HTTP/2. Since generative LLM inference is a one-way system from server to client after prompt ingestion, using Server-Sent Events provides a great fit with minimal complexity to maintain connections. It also provides a way to use the standard EventSource interface [17].

### D. Voice Analytics

Voice analytics provides a good indication of how competent candidates are. Zhang & Liu [2] found there was a strong positive correlation (r = 0.78) between candidates who spoke at a steady pace (120-150 WPM) and high scores from recruiters. However, candidates with a high level of disfluency (> 5%) scored poorly. Voice analytics often requires recording on the server and Python signal processing tools. Career Navigator provides voice analytics in the browser using W3C Web Audio API [16]. This ensures privacy as audio buffers aren't recorded on the server.

### E. Static Analysis

Most automated marking platforms provide dynamic analysis. This means they run tests on the code [27].

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

Kumar & Sharma [5] showed that 43% of inefficiencies in algorithms cannot be detected using dynamic analysis. This includes O(N2) solutions that pass small test cases in time-outs.

Creating an Abstract Syntax Tree allows us to look at algorithms in terms of complexity. This means we can look at the depth of loops, recursive branching and memory allocation without caring about the hardware [28]. Career Navigator uses Abstract Syntax Trees to provide heuristic traversal and generative explanations of code.

## III. SYSTEM ARCHITECTURE AND SECURITY

### A. Architecture

Career Navigator has four layers to its architecture:

1) Frontend (Client Side):

This is written in Typescript and React 19. This provides a responsive UI, Monaco editor, Web Audio DSP and EventSource.

2) Security and API:

This layer is built into Express. It handles incoming requests, CORS, verifying sessions are secure, double-submit CSRF and token bucket IP limiting.

3) Core Services:

This includes:

Document Ingestion – Extracts text from PDF and DOCX files. This also normalises Unicode.

ATS Scoring – Uses DJB2 hashing to detect changes. Then keyword scoring based on categories.

Interview Service – Uses Gemini 2.5 Flash. This is streamed via chunked HTTP/2.

Code Review – Uses AST traversal to find algorithmic complexities in JavaScript/Python.

4) Data Store:

This uses SQLite with Write Ahead Logging. This allows for easy swapping between enterprise RDBMS (e.g. PostgreSQL).

### B. Security

Career Navigator follows OWASP Top 10 [11]. It provides:

Double-submit CSRF – All actions that change state (POST/PUT/DELETE) require a random token to be sent in both the HttpOnly cookie and a custom X-CSRFToken header. If these do not match then an HTTP 403 Forbidden will be sent.

Timing Safe Sessions – Sessions are signed using HMAC-SHA256 [26]. This is checked using constant time comparisons (crypto.timingSafeEqual). This prevents timing attacks.

Rate Limiting – Limiting authentication to 10 attempts per minute per IP using token bucket.

No Voice Data – As part of privacy by design the microphone data will not be sent or stored. All voice analytics will be done in the clients local AudioContext.

## IV. MATHEMATICAL FORMULATIONS

### A. Document Normalisation

When extracting text from PDFs, there will be Unicode characters. This will cause issues with NLP. Let Sraw represent the extracted text. By applying normalisation:

$$S_{norm} = N(S_{raw}) \quad (1)$$

Where N replaces Unicode Ligatures (e.g. fi → fi, fl → fl), converts smart quotes to standard ASCII quotation marks (") and replaces multiple spaces with single spaces.

Career Navigator uses an optimised 32-bit version of the DJB2 hashing algorithm [10]. This gives an 8-character hexadecimal value. This is done quickly (< 0.15ms) for standard 50KB text. If Snorm = c1c2 ...cL then

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

$$H0 = 5381 \quad (2)$$

$$Hi = ((Hi-1 << 5) + Hi-1 + ord(ci)) \% 232 \quad (3)$$

**B. ATS Resume Scoring**

Career Navigator uses a multi-factor scoring system to ensure scoring is deterministic. This provides a score between 40 and 100:

$$SATS = clamp(Sbase + Ssec + Skw + Slen - Pfmt,40,100) \quad (4)$$

Where:

$$Sbase = 50.0 \quad (5)$$

$$Ssec = 0. \quad (6)$$

Where M = {Education,Skills,Projects,Contact} and D is the sections found using regex aliasing:

(7)

$$Skw = min(20.0, Kmatch \times 1.5) \quad (8)$$

Where K is the keyword set and T(Snorm) is the tokenised text. Where Kmatch = |{k ∈ K | k ∈ T(Snorm)}|

$$Slen = +5.0 \text{ if } 350 \le W \le 800 \quad (9)$$

$$Slen = -10.0 \text{ if } W < 200 \quad (10)$$

$$Slen = 0.0 \quad (11)$$

$$Pfmt = 8.0 \times 1\{\neg Email\} + 5.0 \times 1\{\neg Phone\} \quad (12)$$

Algorithm 1 shows the full process for determining the score of a resume using a multi-factor deterministic ATS resume scoring system.

**[t]Algorithm 1 Deterministic Multi-Factor ATS Resume Scoring**

1: Require: Snorm, D, K
2: Ensure: SATS ∈ [40,100]
3: Score ← 50.0
4: M ← {"Education","Skills","Projects","Contact"}
5: Mcount ← |{s ∈ M | s ∈ D}|
6: Score ← Score + (Mcount / |M|) × 25.0
7: T ← Tokenise(Snorm)
8: Kmatch ← 0
9: for all kw ∈ K do
10: if kw.ToLower() ∈ T then
11: Kmatch ← Kmatch + 1
12: end if
13: end for
14: Score ← Score + min(20.0, Kmatch × 1.5)
15: if |T | ≥ 350 and |T | ≤ 800 then
16: Score ← Score + 5.0
17: else if |T | < 200 then
18: Score ← Score − 10.0
19: end if
20: if ¬RegexMatch(Snorm, EmailPattern) then
21: Score ← Score − 8.0
22: end if
23: if ¬RegexMatch(Snorm, PhonePattern) then
24: Score ← Score − 5.0
25: end if
26: SATS ← round(max(40.0, min(100.0, Score)))
27: return SATS

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

**C. Server-Sent Events Generative**

During mock interviews responses will be marked by Google Gemini 2.5 Flash. To give the illusion of less delay during interviews the feedback is streamed using HTTP/2. Let Treq be the time the request is sent, Tfirst be the time the first response is received then:

$$TTFT = Tfirst - Treq \quad (13)$$

The response will be of type text/event-stream. This will disable buffering using CacheControl: no-cache. The response will contain:

$$Frame = \text{"data: "} \,||\, JSON(\{ \text{"token"}: tk \}) \,||\, \text{"\n\n"} \quad (14)$$

Once the generation is complete, "data: [DONE]\n\n" will be sent. This will mean the client closes the connection.

**D. Voice Telemetry**

To provide feedback on communication skills we can look at how fast someone is talking and how many fillers they have. Let Tspoken be the raw text, Nw be the number of words spoken and Dactive be the total amount of time the person was talking (seconds). This can be gathered using the AnalyserNode in Web Audio API. The words per minute:

$$WPM = \text{round} \quad (15)$$

If WPM < 100 then
Status(WPM) = "Too Slow / Hesitant"
Else if 100 ≤ WPM ≤ 150 then
Status(WPM) = "Optimal Cadence"
Else if WPM > 150 then
Status(WPM) = "Too Fast / Rushed"

(16)

The number of fillers can be calculated using regex. Let Fpattern be:

$$Fpattern = \text{\b(um|uh|like|you know|basically|actually)} \quad (17)$$

Then the Disfluency Density Index (DDI) is:

$$DDI = 100\% \quad (18)$$

**E. Static AST Algorithm Complexity**

Instead of looking at run-time, we look at the structure. Let Γ be the root node of the AST of the parsed program. We can look at:

$$dmax = \max_{v \in \Gamma loops} NestingDepth(v) \quad (19)$$

Where Γloops is all the nodes that are looping (ForStatement, WhileStatement, DoWhileStatement).

We can determine if there are logrithmic divide and conquer algorithms by:

$$\delta halv = 1\{\exists u \in \Gamma | u \text{ performs binary index halving}\} \quad (20)$$

We can determine how much memory is used by looking at heap memory allocations:

$$\alpha mem = 1\{\exists w \in \Gamma | w \text{ allocates dynamic data structures}\} \quad (21)$$

Algorithm 2 shows how we can determine algorithmic complexities.

**[t]Algorithm 2 Static AST Big-O Complexity**

1: Require: P, L
2: Ensure: Ot, Os
3: Γ ← ParseSourceToAST(P,L)
4: dmax ← 0, δhalv ← False, αmem ← False
5: for all Node ∈ Traverse(Γ) do
6: if Node.type ∈ { 'For','While','DoWhile' } then
7: d ← ComputeNestingDepth(Node)
8: dmax ← max(dmax,d)

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

9: end if
10: if IsHalvingOperation(Node) then
11: δhalv ← True
12: end if
13: if AllocatesDynamicMemory(Node) then
14: αmem ← True
15: end if
16: end for
17: if δhalv and dmax == 0 then
18: Ot ← O(logN)
19: else if δhalv and dmax == 1 then
20: Ot ← O(N logN)
21: else if dmax == 1 then
22: Ot ← O(N)
23: else if dmax == 2 then
24: Ot ← O(N2)
25: else if dmax >= 3 then
26: Ot ← O(Ndmax)
27: else
28: Ot ← O(1)
29: end if
30: if αmem then
31: Os ← O(N)
32: else
33: Os ← O(1)
34: end if
35: return {Ot, Os}

## V. EXPERIMENTAL ANALYSIS AND PERFORMANCE

### A. Experimental Configuration

Our system was evaluated across the below testbed:

- Server configuration: Dual 20-core 2.20GHz Intel Xeon Silver 4210 CPUs, 64GB DDR4 ECC RAM, Ubuntu 22.04 LTS. Node.js v20.12 LTS runtime.

- Client configuration: 50 computers running Intel Core i5-1135G7 CPUs with 8GB RAM. Chrome v124 and standard USB headset.

- Load Simulation: Apache JMeter v5.6.3 and Artillery v2.0 for simulated user loads.

### B. Inference Latency

To determine whether our system would provide low-latency, we compared how fast the underlying Server Sent Events architecture performed using a full suite of 50 interview questions. The legacy HTTP baseline provided an average Time to First Token (TTFT) of 1580ms. Alongside this, average Tokens per Second (TPS) was 22, resulting in a total inference time of 2150ms. Our system improved this performance significantly, reducing the TTFT to 342ms (78.4%), and meeting our SLA of being under 400ms. Not only did we drop the TTFT, we increased the average TPS to 54 (145.5% improvement) and reduced the total inference time by 15.3% to 1820ms. Our networking improved similarly, reducing the ram required to store sockets by 66.1% from 12.4KB per client (long polling), improving to 4.2KB per client (HTTP/2 multi-plexed SSE). This greatly improved user experience by no longer putting them in a 'waiting' state using spinners. Instead we could provide them with live feedback.

### C. ATS Score Validity

We conducted a blind experiment using 50 computer science undergraduate resumes, and gained scores from 3 experienced Senior Technical Recruiters working at Enterprise Software companies. They scored the candidates

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

based on technicality, formatting and completeness using a 100-point rubric. We took the average to create our gold standard (Shuman). The scores were then categorised into 3 groups:

High Readiness: Scores were between 80-100 (N = 18). Average score was 88.4 from recruiters and 87.1 from ATS. (MAE: 2.1) (Pearson: r = 0.941, p < 0.001)

Moderate readiness: 60-79 (N = 22). Average score was 69.2 from recruiters and 70.8 from ATS. (MAE: 3.2) ( r = 0.885, p < 0.001)

Candidates needing substantial editing: <60 scores (N = 10). Average score was 48.5 from recruiters and 47.2 from ATS. (MAE: 2.8) ( r = 0.912, p < 0.001)

Across the entire cohort scores (N = 50), they scored an average of 72.0 points from recruiters, and 71.9 from ATS (MAE: 2.7). Using linear regression we found this had a Pearson Correlation Coefficient of N = 220__, giving us a high determination of R2=0.892 (N = 221__). This shows our deterministic method mimics expert human rubrics well.

**D. Audio Telemtry**

The final backend module we tested was our Web Audio Pacing Checker, using 50 audio samples. Our objective was to compare manual annotation of samples against our Web Audio. Job seekers spoke an average of 128.4 words per minute (WPM) (Manual GT). However, our browser gave an average of 127.1 WPM. Overall this gave us a high accuracy of 98.9%, and only had a small variance of +/-.3 WPM. We also tested how our program handled disfluencies in the audio. Humans identified 34 fillers (um). Our regex engine was able to find 32 of these correctly (94.1%). We also found, 45 of the 48 colloquial fillers (like) were found by our engine (93.8%). Our final audio length was also deemed accurate (179.2 vs 180.0 seconds, 99.5% accuracy). Overall we have achieved a high level of accuracy using the Digital Signal Processing features in the browser.

**E. Big-O Complexity Calculation**

To ensure our static Big-O calculator was performing well, we ran it against 40 algorithmic examples written in JavaScript and Python. Linear algorithms (O(N ), N = 222__) were found with 100% accuracy (10/10), and had 90% edge cases identified. Divide and Conquer algorithms (O(log N ), e.g. Binary Search, N = 223__) were also found well, with 100% accuracy (10/10) time complexity and 90% (9/10) space complexity. Edge cases were identified with 80% accuracy (8/10). Quadratic algorithms using nested loops (O(N2 ), e.g. Bubble Sort, N = 224__) had 100% accuracy (10/10) both time and space complexity. Edge cases were found with 100% accuracy (10/10). Recursive Tree structures (O(2N ), and branch traversals, N = 225__) were found with 80% accuracy (8/10) time and space. They had 70% edge cases identified, due to dynamic pruning of branches. Across all problems our calculator was correct 95% (38/40) and had 92.5% (37/40) space complexity with 85% edge cases identified.

**F. Concurrency Performance**

To ensure we were able to scale well across enterprise instances, we used Apache JMeter and Artillery to simulate user traffic. With 50 test users across 5000 requests, our system had a low average latency of 42ms (95th percentile, P95=85ms ), with a throughput of 118 requests and 0.00% errors. This improves as we increase users to 100 across 10,000 requests. This increased the latency to 68ms (P95=142ms ), but improved the throughput to 230. Under highload of 250 users and 25,000 requests the average response time did not change much, rising to 125ms (P95=260ms ). Throughput improved to 485 with only 0.02% errors. We saw the highest improvement with 500 users across 50,000 requests. Our system had a high throughput of 720 requests. The average response time was 285ms (P95=590ms) with 0.15% errors. This shows we would be able to scale well for deployment across campuses.

**G. User Experience Measurement**

To understand how candidates viewed our system, we ran a SUS against 50 final year Computer Science undergraduates. Following 45 minutes of preparation, we received an average score of 86.4 out of 100. This puts Career Navigator in the top 95th percentile (Grade A+ Exceptional).

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
( Peer-Reviewed, Open Access, Fully Refereed International Journal )
Volume:08/Issue:03/March-2026 Impact Factor- 8.187 www.irjmets.com

## VI. DISCUSSION, EDUCATIONAL IMPACT AND LIMITATIONS

### A. Explaining how our platform avoids black-box syndrome

Pre-screens are often criticised for not providing transparency as to how they are ranking candidates [8]. This happens due to the black-box nature of neural networks. Due to this they can pick up on proxy factors within hiring data for specific demographics [7][25]. This issue is avoided with our deterministic ATS technique (Algorithm 1). Our golf focuses purely on technical keywords, section completion and text formatting used. This means every point loss has an actionable explanation.

### B. Educational Benefit of real-time feedback

Due to our great low inference time we are able to create an interactive conversation. During piloting, we found that candidates who received real-time pacing information changed their speech to fall into the ideal range (120-140WPM) from an anxious rate (>175WPM). This was done after only an average of 3 practice sessions. The static AST tool also encouraged candidates to optimise their quadratic solutions.

### C. Limitations

Although we obtained great results, there are some limitations to our work:

Not all accents and dialects are equally accurate. Whilst the client-side Speech Recognition API is accurate, the accuracy can vary depending on the regional accent of non-native English speakers and tech dialcts. This can result in trouble identifying jargon.

Static analysis cannot determine dynamic run-time variables. Whilst the static AST analysis is accurate for identifying standard looping algorithms, it cannot calculate when the algorithm bounds depend on variables provided during run-time. Example: Quicksort pivot choice (randomised) and pruned branches during backtracking.

Dependencies on 3rd party foundation models. If the Google Gemini API goes down then we will not get the maximum possible features during interviews. We have created heuristic simulators to help keep our system online, however we cannot generate dynamic out-of-script questions..

## VII. CONCLUSION AND FUTURE WORK

This paper describes the design, development and experimental results of Career Navigator. It delivers a form of enterprise grade pre-screening by normalising the resumes using deterministic unicode techniques, using low latency ServerSent Events for tokens, Web Audio modules to analyse speech and static AST analysis to calculate Big-O complexity. When using it against 50 engineering resumes we saw great correlation (R2=0.892, N = 226__) with Senior Corporate Recruiters. Our system used speech telemetry to give great accuracy (98.9%) whilst not sending raw audio. We also saw our system could sustain 720 requests when there were 500 test users.

We intend to explore future work using more advanced techniques including:

Including Small Language Models (SLMs) using WebAssembly( Wasm ). These models will run on device on the client-side and will have access to WebGPU to support private reasoning during conversation.

Incorporating Computer Vision models using WebRTC. This will allow us to monitor candidates eye contact and body language during mock interviews.

## VIII. REFERENCES

[1] M. Anderson, L. Roberts, and K. Chen, “Automated information extraction and entity parsing of irregular resumes using conditional random fields,” IEEE Trans. Knowl. Data Eng., vol. 33, no. 5, pp. 2145-2158, May 2021.

[2] X. Zhang and Y. Liu, “Audio and prosodic features for confidence and fluency in video interviews using automated systems,” in Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP), Singapore, 2022, pp. 7832-7836.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, 2017, pp. 5998-6008.

[4] T. Brown et al., “Language models are few-shot learners,” in Adv. Neural Inf. Process. Syst. (NeurIPS), Dec.

2020, pp. 1877-1901.

[5] S. Kumar and R. Sharma, "Investigating the quality and efficiency of algorithmic code using abstract syntax trees in automated assessment platforms," ACM Trans. Comput. Educ., vol. 23, no. 2, pp. 112-129, Jun. 2023.

[6] M. Chen et al., "Evaluating large language models trained on code," arXiv, 2022, arXiv:2107.03374.

[7] A. Patel and V. Deshmukh, "Avoiding bias-in-algorithms and disparate impact in machine learning-based recruitment screening systems," in Proc. ACM Conf. Fairness, Accountability, Transparency (FAccT), 2023, pp. 412-424.

[8] C. O'Neil, Weapons of Math Destruction: How Big Data Increase Inequality And Threaten Democracy . New York, NY, USA: Crown Publishing, 2021.

[9] T. Radboud, K. Van Dijk, and S. De Vries, "Performance benchmarks of Server-Sent Events and WebSockets for token-streaming in genertive models," IEEE Internet Comput., vol. 26, no. 4, pp. 45-53, Jul./Aug. 2022.

[10] A. Gupta, S. Murthy, and P. Verma, "High-performance duplicate detection solutions using 32-bit DjB2 variation for high velocity microservices," in Proc. IEEE Int. Conf. Cloud Eng. (IC2E), 2023, pp. 189-197.

[11] OWASP Foundation, "OWASP Top Ten Security Risks Conflicts," Tech. Rep. OWASP-2023-V1.2, Nov. 2023. [Online]. Available: https://owasp.org/www-project-top-ten/

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Conf. N. Amer. Chapter Assoc. Comput. Linguists. (NAACL-HLT), 2019, pp. 4171-4186.

[13] Gemini Team, "Gemini: A family of highly capable multi-modal models," Tech. Rep. Google Research, December 2023. [Online]. Available: https://arxiv.org/abs/2312.11805

[14] Gemini Team, "Gemini 1.5: Multi-modal understanding across millions of tokens of context," Tech. Rep. Google Research, March 2024. [Online]. Available: https://arxiv.org/abs/2403.05530

[15] M. Fowler, and K. Beck, Refactoring: Improving The Design Of Existing Code, 2nd Ed. Boston, MA: Addison-Wesley, 2019.

[16] World Wide Web Consortiums Audio Working Group, "Web audio API: W3C candidate recommendation,", Jun. 2021. [Online]. Available: https://www.w3.org/TR/webaudio/

[17] Mozilla Developer Network, "Server-sent events overview," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent events

[18] Microsoft Corporation, "Monaco Editor," 2024. [Online]. Available: https://microsoft.github.io/monacoeditor/

[19] R. Fielding, Architectural Styles And The Design Of Network-Based Software Architectures , Ph.D. Thesis, Univ. California, Irvine, CA, 2000.

[20] Y. Xu et al., "LayoutLM: Pre-training of text and layout for document image understanding," in Proc. ACM SIGKDD Int. Conf. Know. Discov. Data Min., 2020, pp. 1192-1200.

[21] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv, 2023, arXiv:2303.12712.

[22] M. Resnick, B. Myers, and K. Ko, "Computer science education with automated formative feedback: A meta-analysis," ACM Comput. Surv., vol. 52, no. 4, pp. 78:1-78:34, 2019.

[23] R. Nallapati, B. Zhou, C. Gulcehre, and B. Xiang, "Abstractive text summarisation with sequence-to-sequence RNNs and beyond," in 20th SIGNLL Conf. Comput. Nat. Lang. Learn. (CoNLL), Berlin, Germany, 2016, pp. 280-290.

[24] P. Henderson et al., "Progress towards controllable, low latency generative conversation models," in Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP), 2020, pp. 8154-8158.

[25] D.F. Mujtaba and N.R. Mahapatra, "Responsible AI-based recruitment: Challenges, metrics, approaches and the way ahead," arXiv, 2024.

[26] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS PUB 180-4,

Federal Information Processing Standards Publication, Aug. 2015.

[27] J. C. Paiva, J.P. Leal, and ´A. Figueira, "Automated assessments in computer science education: State of the art review," ACM Comput. Surv., vol. 54, no. 11s, pp. 234:1-234:40, 2022.

[28] A. V. Aho, M.S. Lam, R. Sethi and J.D. Ullman, Compilers: Principles, Techniques, And Tools , 2nd Ed. Boston, MA: Addison-Wesley, 2006.

[29] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction To Algorithms , 4th Ed. Cambridge, MA: MIT Press, 2022.

[30] R.C. Martin, Clean Architecture: A Craftsman's Guide To Software Structure And Design . Boston MA: Prentice Hall,